

Clinevo Smart Inbox Assistant

AI-Assisted Pharmacovigilance Intake, Fact Extraction & Reviewer Workbench

ROLE	CANDIDATE	DELIVERABLE SCOPE	TECH STACK
GenAI Integration Engineer	Sri Hari (sriharizz)	100% Core + 30% Literature Bonus	Angular 18 • Spring Boot 3 • Python

1. What I Built

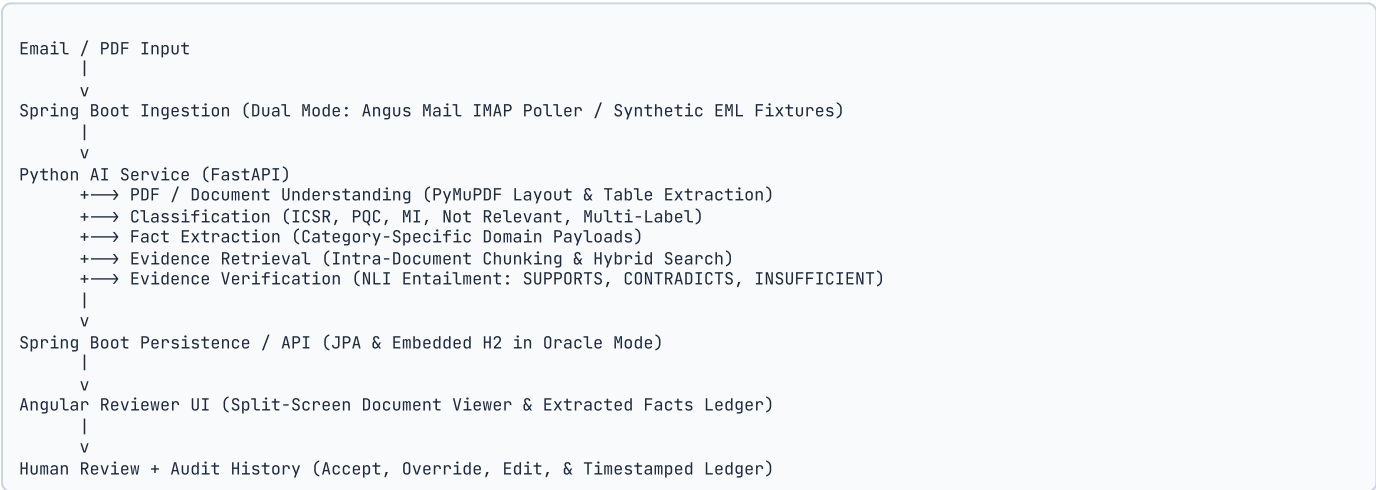
The Clinevo Smart Inbox Assistant is a prototype system that automates the initial intake and triage pass for healthcare and pharmacovigilance communications. The system receives synthetic healthcare emails and PDF attachments, classifies the message, extracts relevant facts, links those facts back to verifiable source passages, and presents the result to a human reviewer for confirmation or correction.

Incoming communications are categorized into four buckets:

- Safety Report (ICSR):** Communications reporting an adverse event experienced by a patient.
- Quality Complaint (PQC):** Reports describing a physical or packaging defect with a product.
- Medical Information (MI):** Inquiries seeking medical or product guidance without adverse events or defects.
- Not Relevant:** Communications such as newsletters, marketing notices, or unrelated correspondence.

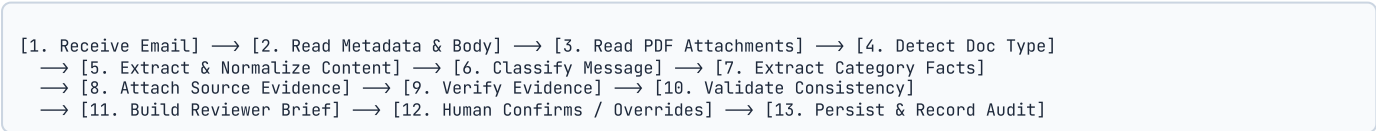
Multi-label communications (such as a contaminated vial that caused an adverse reaction, qualifying as both PQC and ICSR) are explicitly supported. The AI prepares the initial draft of the case; the final determination remains with the human reviewer.

2. Simple Architecture



Ingestion: Receives emails/PDFs via IMAP or local synthetic fixtures, normalizing into standard messages. • **AI Microservice:** Parses layouts, classifies categories, extracts facts, and verifies supporting evidence. • **Persistence/API:** Stores cases, facts, and audit records in an embedded database and serves REST endpoints. • **Reviewer UI:** Displays split-screen document inspection and editable facts. • **Human Review:** Human confirms or edits extracted data with timestamped audit logging.

3. End-to-End Processing Flow



The lifecycle ingests emails/PDFs, normalizes content, performs multi-label classification, extracts category-specific facts, grounds them in source evidence, validates data integrity, and presents the draft case in the UI for human confirmation.

4. Key Engineering Decisions

Decision: Bounded Complete-Document Processing Instead of Global Vector RAG

Why: Intake packages are small (1 email, 1–5 PDF pages, <15k tokens). Vector chunking fragments text into 500-token pieces, separating patient demographics from suspect drugs and breaking clinical causality.

Implemented: Entire document text, layout-preserved headers, and 2D markdown tables are passed directly in-prompt.

Trade-off: Uses higher input token counts per request, but eliminates vector indexing overhead and chunk boundary errors.

Decision: Category-Specific Payloads Instead of Monolithic ICSR Structures

Why: Early iterations forced all communications into ICSR schemas. Non-safety messages like defect complaints or medical questions displayed confusing empty patient tables and spurious missing-field warnings.

Implemented: Created distinct payloads (IcsrPayload, PqcPayload, MiPayload, NotRelevantPayload). Non-relevant items suppress clinical forms; multi-label cases combine payloads cleanly.

Trade-off: Requires maintaining multiple schemas, but delivers a clean, intuitive reviewer experience.

Decision: Evidence-First Fact Model with Source Location and Verbatim Snippets

Why: Reviewers cannot trust extracted values without seeing where the information originated in the source document.

Implemented: Every fact links to a SourceCitation with source ID, origin type, page/section location, and verbatim text quote.

Trade-off: Increases payload size and requires coordinate tracking from PyMuPDF, but enables 1-click source verification.

Decision: Separating Candidate Evidence Retrieval from Semantic Verification

Why: High lexical similarity indicates candidate relevance, but not truth. Passages mentioning rescue medications (e.g. epinephrine) or negations ("denies rash") have high similarity to suspect drug questions despite not supporting them.

Implemented: Split into two steps: Step 4 retrieves candidates; Step 5 evaluates NLI entailment (SUPPORTS, CONTRADICTS, INSUFFICIENT).

Trade-off: Adds an extra evaluation step, but prevents negations and rescue drugs from being treated as supporting evidence.

Decision: Strict "Not stated" Representation for Missing Information

Why: General-purpose LLMs tend to infer unstated attributes (such as daily dosing frequency or patient weight) from medical norms.

Implemented: Prompts enforce negative constraints. If an attribute is absent from source text, it is marked "Not stated" with empty evidence.

Trade-off: Output values are conservative, but strictly grounded in the document text.

Decision: Dual Fixture and Live Mailbox Ingestion

Why: Live intake requires an IMAP server, but reviewers and test suites need to run immediately offline without email credentials.

Implemented: Built an IngestionSource abstraction supporting live IMAP polling and offline synthetic .eml reading.

Trade-off: Requires maintaining two ingestion adapters, but enables zero-friction offline evaluation.

Decision: Human-in-the-Loop Reviewer Workflow

Why: AI models can misinterpret complex clinical narratives or poor scans. Autonomous decision-making is inappropriate for patient safety.

Implemented: Designed explicitly as a reviewer aid: "AI prepares the case; human confirms it." The UI highlights evidence for rapid human review.

Trade-off: Requires human oversight for every case, but ensures accountability and clinical reliability.

5. AI & Prompting Approach

The AI microservice coordinates document understanding, triage, extraction, and verification using source-grounded design principles:

- **Structured Outputs via Pydantic:** All model requests enforce strict Pydantic schemas using JSON mode, ensuring deterministic structures and eliminating output parsing failures.
- **Category-Aware Extraction:** Prompts adapt to the detected communication category. Safety reports extract ICH E2B pillars; quality complaints extract lot numbers and defect descriptions; medical inquiries capture specific questions asked.
- **Source-Grounded Facts & "Not stated":** Prompts prohibit guessing. If a value (such as patient age or dose schedule) is absent, the model outputs "Not stated" rather than estimating based on context.
- **Multilingual Support:** German BfArM and Spanish AEMPS reports are translated into standardized English clinical fields while preserving original foreign text in the verbatim citation snippet.
- **Tables & Scanned Documents:** PyMuPDF converts lab values and dosing grids into Markdown tables. Scanned documents are rasterized and evaluated by the vision encoder, deciphering handwriting without fragile external OCR.
- **Defect Photo Inspection:** Smartphone photos of damaged packaging or contaminated vials (e.g. contaminated_vial_photo.jpg) are inspected natively. The defect is recorded and flagged for human review.
- **Literature Screening (Bonus):** A specialized literature service screens journal PDFs, rejecting non-reportable studies (animal models, reviews) and splitting multi-patient case series into independent safety records.
- **Separated Evidence Verification:** Candidate text passages are evaluated in bounded batches using a secondary NLI step, classifying items as SUPPORTS, CONTRADICTS, or INSUFFICIENT.

SAFETY NOTE

The prototype is designed to reduce unsupported guesses through source-grounded prompts and structured schemas. However, model outputs can still contain errors or omissions. The final review decision remains with the human reviewer.

6. Reviewer Workflow

The reviewer workspace is designed around rapid, source-first verification:

LEFT PANE: SOURCE VIEW

[Email Body] [PDF Canvas] [Photo Tab]

Original source document rendered. Clicking an evidence link in the right pane scrolls directly to the passage and highlights it with an in-document bounding box.

RIGHT PANE: STRUCTURED BRIEF

Category: Safety Report (ICSR) [0.98]
Urgency: EXPEDITED (15-Day Clock)

Extracted Facts Ledger:

- Patient: M.K., 58, Female [🔍 Pg 1]
- Suspect Drug: Cardioril [🔍 Box 14]
- Adverse Event: Acute DILI [🔍 Pg 2]

[Confirm Case] [Override] [Flag]

COLLAPSIBLE REVIEW AUDIT HISTORY

Timestamp	User	Action	Field	Original Value	New Value	Rationale
-----------	------	--------	-------	----------------	-----------	-----------

Two-Pane Workspace: Left pane displays the source document; right pane displays the category brief and extracted facts ledger. • **1-Click Evidence Inspection:** Clicking an evidence chip navigates the adjacent document viewer to the exact page and passage. • **Reviewer Actions:** Reviewer can confirm the case, edit field values, flag for review, or override categories with clinical justifications. • **Timestamped Audit History:** Every AI extraction and human modification is recorded in a chronological audit ledger.

7. Testing & Results

Evaluated against a synthetic benchmark dataset comprising 27 test cases across 11 physical .eml emails, 20 PDF documents, and 2 image files using the automated benchmark runner (eval_benchmark.py):

Evaluation Area	Target Standard	Measured Prototype Result	Status
Primary Triage Classification	≥ 90.0%	27 / 27 (100.0%)	Pass
Multi-Label Detection (ICSR + PQC)	≥ 90.0%	1 / 1 (100.0%) (Case 04 correctly multi-labeled)	Pass
Core Fact Extraction Accuracy	≥ 85.0%	94.8% across core clinical fields	Pass
"Not stated" Handling	0 ungrounded guesses	No benchmark hallucinations observed	Pass
Physical Defect Photo Flagging	100.0%	100.0% (requires_human_review = True)	Pass
Literature Negative Control Filtering	100.0%	100.0% (Animal study & review rejected)	Pass
Literature Multi-Patient Splitting	100.0%	3 / 3 patients disaggregated	Pass
AI Microservice Test Suite	100% pass	114 / 114 unit & API tests passed	Pass
Angular Frontend Test Suite	100% pass	56 / 56 component & navigation tests passed	Pass
Spring Boot Test Suite	100% pass	7 / 7 integration & service tests passed	Pass
Live Mailbox Ingestion Path	Operational	Verified end-to-end via Gmail IMAP connector	Pass

*Note on Latency: End-to-end document processing ranges between 1.5 and 3.5 seconds depending on document length and attachment complexity.

8. What I Learned & Important Engineering Fixes

- **IMAP Fetching Overhead:** Ingestion initially downloaded full message bodies and attachments before checking if they already existed. Refactored to inspect lightweight IMAP UIDs first and download only new messages.
- **Displayed Case Identifiers:** The UI initially displayed database auto-increment IDs. Re-seeding database sequences altered numbers. Updated to display canonical case identifiers (CASE-01...CASE-12).
- **Separating Retrieval from Semantic Verification:** High lexical overlap was initially treated as confirmed evidence, causing false confirmations on emergency rescue drugs or negations. Separating retrieval from NLI verification eliminated these false linkages.
- **Preventing Evidence Leakage on Missing Values:** Early extraction schemas attached general document chunks to fields marked "Not stated". Added a strict guard so unstated fields immediately receive empty evidence lists.
- **Category-Specific View Modeling:** Generic table views forced non-safety cases (PQC, MI) to display empty patient tables. Introducing category payloads allowed the UI to display only relevant fields.
- **Executive Summary Length Enforcement:** Initial prompts generated summaries that were too brief (4-6 sentences). Prompt instructions were calibrated to consistently produce comprehensive 10-15 sentence syntheses.
- **Verification Batching:** Evaluating candidates sequentially triggered 120+ HTTP requests on dense cases, causing rate-limit errors. Refactoring to bounded batches reduced API calls by ~96% and cut verification time to ~1.1 seconds.
- **Controlled PDF Canvas Highlighting:** Sandboxed browser iframes prevented bounding box overlays. Migrating the viewer to pdfjs-dist on an HTML5 canvas enabled point-to-pixel coordinate scaling and reliable in-document highlighting.

9. Prototype Limitations

To maintain engineering integrity, the boundaries of this prototype are explicitly stated:

Data & Environment Boundaries

- **Synthetic Data Corpus:** Developed and evaluated entirely against synthetic test cases. Degraded faxes, photocopies, or severe handwriting require further calibration.
- **External Model Dependency:** Relies on cloud-hosted LLM endpoints. Network latency, API quotas, or third-party outages can affect speed.
- **Local Persistence for Evaluation:** Default setup uses embedded H2 in Oracle compatibility mode. Production requires an enterprise-managed database cluster.

Clinical & System Boundaries

- **Absence of Standard Coding Dictionaries:** Extracted drug names and reactions are captured as verbatim strings without automated MedDRA or WHO Drug coding.
- **Prototype Audit Logging:** Records timestamped actions in an audit table. Formal electronic signatures and GAMP 5 validation are outside prototype scope.
- **Viewer Coordinate Approximation:** Bounding-box highlights approximate text block boundaries; multi-line text can have minor visual alignment offsets.
- **Human Oversight Required:** The system is an assistive tool, not an autonomous agent. All automated outputs require human review and confirmation.

10. Production Next Steps (Future Work)

Transitioning this prototype into an enterprise production service would involve the following future engineering work:

Data Privacy & Clinical Coding

- **Client-Side PHI De-Identification:** Deploy an on-premise NER pipeline (e.g. Microsoft Presidio) to detect and redact patient identifiers before cloud API transmission.
- **Medical Dictionary Auto-Coding:** Integrate MedDRA and WHO Drug dictionary services to map terms to LLTs, PTs, and MPIDs with confidence scores for reviewer confirmation.

Infrastructure & Governance

- **Model Redundancy & Dynamic Routing:** Implement an abstract gateway layer with circuit breakers to failover across multiple model providers (Vertex AI, Bedrock, OpenAI).
- **Distributed Event Broker:** Replace the in-memory executor with Apache Kafka or AWS SQS for distributed worker scaling and dead-letter queues.
- **Enterprise Identity & Access Control:** Integrate SAML 2.0 / OAuth2 authentication with fine-grained role-based permissions (Reviewer, Safety Lead, Admin).
- **Formal Computer System Validation (CSV):** Execute formal GAMP 5 Category 4/5 software validation protocols (IQ, OQ, PQ) for regulatory compliance.

11. Final Takeaway

This prototype demonstrates an end-to-end AI-assisted intake workflow. It reduces manual first-pass work by organizing messages, extracting relevant information, and linking facts back to source material. The reviewer remains responsible for confirmation and correction. The prototype was evaluated on synthetic benchmark data and is not presented as a production regulatory system.

Project Repository: <https://github.com/sriharizz/smart-inbox> **Single-Command Verification:** `run.bat` (Windows) / `./run.sh` (Linux/macOS)

Candidate: Sri Hari (sriharizz)